PART 2B: Core Backend Logic & Infrastructure

Data Collection, Telemetry Ingestion, Deterministic Safety Governor, SQLite Persistence & Communication Logic

Author: Saptarshi Das (VIT Vellore)
Email: saptarshid234@outlook.com
Repository: saptarshidas578/smartgrid_ai

Primary Modules: safety_engine.py, database.py, mqtt_client.py
Infrastructure: config.py, mock_esp32.py, run_smart_grid_suite.py
Role: Deterministic Safety, Data Collection & System Coordination

1. Executive Overview: The Deterministic Safety Cage

Part 2B encapsulates the non-AI infrastructure and deterministic control logic of the smart grid. It is responsible for **sub-second data collection**, **hard physical safety enforcement**, **ACID database persistence**, and **bidirectional MQTT networking**. In this architecture, all machine learning predictions from Part 1 are treated strictly as untrusted advisories: the Safety Governor verifies every predicted trajectory against physical Kirchhoff's current laws, source ratings, and switchgear anti-chatter timers before any relay command can reach physical field hardware.

Subsystem	Core Module	Governing Mechanism	Design Constraint / Guarantee
Safety Governor	safety_engine.py	Priority Knapsack Shedding Algorithm	Trips at ≥85% utilization; restores at ≤84%; strictly immunizes Critical load
Anti-Chatter Timer	safety_engine.py	15.0s Dwell-Time State Machine	Prevents relay contact welding and destructive high-frequency mechanical chatter
Data Collection	mqtt_client.py	Paho MQTT TLS 1.3 Client	Ingests 1 Hz JSON telemetry on smartgrid/live; dispatches commands with QoS 1
Database Engine	database.py	SQLite in Write-Ahead Logging (WAL)	Zero-lock multi-threaded reads/writes; indexed B-tree temporal lookups
Configuration	config.py	Central Dataclass Store	Single source of truth for nominal ratings, thresholds, priority ranks, and credentials
Virtual Grid Mock	mock_esp32.py	Gaussian Noise Physical Model	Replicates INA219 shunt sensors and relay actuation for offline CI/CD testing
Supervisor	run_smart_grid_suite.py	Multiprocessing Orchestrator	Coordinated startup/shutdown, health checks, and signal handling across all daemons

2. Data Collection & Telemetry Ingestion: src/core/mqtt_client.py

Section 2.1: Resilient TLS MQTT Client Engine & Thread-Safe Publishing

```
12: class ResilientMQTTClient:
13:     def __init__(self, broker: str, port: int, username: str, password: str, client_id: Optional[str] = None):
26:         self.connected = False
27:         self._lock = threading.Lock()  # Mutex for socket thread safety
35:         self.client.username_pw_set(self.username, self.password)
36:         self.client.tls_set()  # Enforce TLS 1.2/1.3 encryption on Port 8883
42:     def _on_connect(self, client, userdata, flags, rc, *args):
43:         if rc == 0:
44:             self.connected = True
46:             client.subscribe([(config.mqtt_topic_live, 0), (config.mqtt_topic_control, 1)])
54:     def _on_message(self, client, userdata, msg):
57:         payload = json.loads(msg.payload.decode("utf-8"))
58:         if msg.topic == config.mqtt_topic_live and self.on_telemetry_callback:
59:             self.on_telemetry_callback(payload)  # Routes to Data Ingestion Pipeline
65:     def publish_command(self, payload: dict):
66:         with self._lock:  # Mutex locked dispatch
67:             self.client.publish(config.mqtt_topic_control, json.dumps(payload), qos=1)
```

Lines 12–36: Establishes authenticated, TLS-encrypted connection with HiveMQ Cloud over port 8883.

Lines 42–46: Subscribes to 1 Hz telemetry uplink (smartgrid/live, QoS 0) and actuator feedback (smartgrid/control, QoS 1).

Lines 54–59 (Data Collection Ingestion Hook): Ingests raw JSON packets, decodes binary payloads into Python dictionaries, and triggers the asynchronous telemetry ingestion pipeline.

Lines 65–67: Implements mutex-locked command publishing, preventing race conditions between concurrent automated shed commands and manual operator web clicks.

3. Detailed Dissection: src/services/safety_engine.py

Section 3.1: Hardware 2-Sigma Breaker & Capacity Overload Shedding

```
36:     def evaluate_telemetry(self, row: dict, publish_cmd_fn) -> Tuple[bool, float, str]:
47:         limit = float(row.get("current_limit", 5000.0))
48:         total_current = float(row.get("total_current", 0.0))
57:         utilization = (total_current / limit * 100.0) if limit > 0 else 0.0
60:         # 1. Instant 2-SD Overcurrent Protection
61:         for load_name in ["hospital", "street", "house", "industry"]:
62:             curr = float(row.get(f"{load_name}_current", 0.0))
63:             thresh = config.statistical_2sd_thresholds.get(load_name, 999.0)
64:             if curr > thresh and self.commanded_relays.get(load_name) == 1:
65:                 self._shed_load(load_name, f"LOCAL TRIP (Curr {curr:.1f}mA > 2SD {thresh}mA)", publish_cmd_fn)
75:         # 2. Priority Knapsack Shedding at >= 85% Utilization
76:         if utilization >= 85.0:
78:             active_sheddable = [l for l in ["industry", "house", "street"] if self.commanded_relays.get(l) == 1]
79:             active_sheddable.sort(key=lambda x: config.load_priorities.get(x, 1))
80:             for load_name in active_sheddable:
82:                 if (now - self.relay_turn_on_time.get(load_name, 0.0)) >= config.anti_chatter_dwell_sec:
83:                     self._shed_load(load_name, f"CAPACITY SHED ({utilization:.1f}% >= 85%)", publish_cmd_fn)
84:                     break  # Step-by-step shedding
```

Lines 60–65: Evaluates sector currents against static 2σ thresholds ($320\text{ mA}, 272\text{ mA}, 405\text{ mA}, 256\text{ mA}, 264\text{ mA}$), immediately issuing a shutoff command if violated.

Lines 75–79 (Knapsack Priority Shedding): Triggers when utilization reaches $\ge 85\%$. Sorts sheddable loads by priority ascending: Industry (P1) $\to$ House (P2) $\to$ Street (P3). Hospital (P4) and Critical Infrastructure are omitted, guaranteeing life-safety immunity.

Lines 80–84: Verifies the 15-second anti-chatter dwell time before commanding de-energization.

Section 3.2: Hysteretic Knapsack Restoration & Anti-Chattering State Machine

```
100:         # 3. Hysteretic Opportunistic Restoration at <= 84% Utilization
101:         elif utilization <= 84.0 and not in_startup:
103:             inactive_loads = [l for l in ["street", "house", "industry"] if self.commanded_relays.get(l) == 0]
104:             inactive_loads.sort(key=lambda x: -config.load_priorities.get(x, 1))
105:             for load_name in inactive_loads:
107:                 if (now - self.relay_turn_off_time.get(load_name, 0.0)) >= config.anti_chatter_dwell_sec:
108:                     nom_draw = config.nominal_draws.get(load_name, 250.0)
110:                     if (total_current + nom_draw) <= (0.84 * limit):
111:                         self._restore_load(load_name, f"RESTORE ({utilization:.1f}% <= 84%)", publish_cmd_fn)
112:                         break
140:     def validate_manual_switch(self, load: str, state: int) -> Tuple[bool, str]:
145:         last_time = self.relay_turn_on_time.get(load, 0.0) if state == 0 else self.relay_turn_off_time.get(load, 0.0)
146:         elapsed = time.time() - last_time
147:         if elapsed < config.anti_chatter_dwell_sec:
148:             return False, f"Anti-chatter lockout: wait {config.anti_chatter_dwell_sec - elapsed:.1f}s"
149:         return True, "Allowed"
```

Lines 100–104: Restores loads when utilization drops below 84%. Evaluates inactive loads in descending order of priority, restoring higher-priority feeders first.

Lines 108–112: Predictive headroom check: confirms $(I_{\text{tot}} + \bar{I}_j) \le 0.84 \cdot I_{\max}$ before restoring, preventing hysteretic on/off hunting oscillations.

Lines 140–149: Validates manual human operator clicks from the web dashboard against the 15-second anti-chatter dwell timer.

4. Detailed Dissection: src/core/database.py (Zero-Lock Persistence)

Section 4.1: SQLite WAL Mode & Concurrency Configuration

```
15:     def get_connection(self) -> sqlite3.Connection:
16:         conn = sqlite3.connect(self.db_path, timeout=10.0)
17:         conn.row_factory = sqlite3.Row
19:         # Enable WAL mode for zero-lock concurrent multi-threaded reads/writes
20:         conn.execute("PRAGMA journal_mode=WAL;")
21:         conn.execute("PRAGMA synchronous=NORMAL;")
22:         return conn
26:     def init_db(self):
30:         cursor.execute("""
31:             CREATE TABLE IF NOT EXISTS telemetry_logs (
32:                 id INTEGER PRIMARY KEY AUTOINCREMENT,
33:                 timestamp TEXT, mode TEXT, current_limit REAL,
36:                 hospital_current REAL, street_current REAL,
37:                 house_current REAL, industry_current REAL, critical_current REAL,
44:                 total_current REAL, total_power REAL,
46:                 predicted_current REAL, predicted_power REAL,
48:                 fault_detected INTEGER, severity TEXT
49:             )
50:         """)
51:         cursor.execute("CREATE INDEX IF NOT EXISTS idx_telemetry_ts ON telemetry_logs (timestamp);")
```

Lines 15–22 (Zero-Lock WAL Architecture): Standard SQLite locks the entire database file during writes. Enabling Write-Ahead Logging (PRAGMA journal_mode=WAL) permits concurrent readers (Streamlit, WebSockets) to query data simultaneously while the 1 Hz telemetry loop writes new records.

Lines 30–51: Defines schema with explicit B-tree index on the timestamp column, ensuring sub-millisecond query execution over 10,000+ historical records.

5. Detailed Dissection: src/core/config.py & Lifecycle Supervision

Section 5.1: Central Parameter Registry & Multiprocessing Supervisor

```
# config.py
15: class Config:
16:     nominal_draws = {"critical": 300.0, "hospital": 256.0, "street": 394.0, "house": 241.0, "industry": 240.0}
17:     statistical_2sd_thresholds = {"critical": 320.0, "hospital": 272.0, "street": 405.0, "house": 256.0, "industry": 264.0}
18:     load_priorities = {"hospital": 4, "street": 3, "house": 2, "industry": 1}
19:     source_limits = {"DUAL SMPS": 5000.0, "SINGLE SMPS": 1000.0, "BATTERY": 800.0}
20:     anti_chatter_dwell_sec = 15.0
21:     startup_observe_sec = 30.0

# run_smart_grid_suite.py
45: def run_suite():
46:     procs = [
47:         subprocess.Popen([sys.executable, "src/smart_grid_ai.py"]),
48:         subprocess.Popen([sys.executable, "-m", "uvicorn", "src.dashboard.grid_dashboard:app", "--port", "8000"])
49:     ]
55:     signal.signal(signal.SIGINT, lambda s, f: [p.terminate() for p in procs])
```

Lines 15–21 (config.py): Central dataclass storing nominal electrical draws, 2σ trip limits, priority weights, and timing constraints.

Lines 45–55 (run_smart_grid_suite.py): Master process supervisor managing coordinated startup, health monitoring, and graceful termination of the telemetry daemon and web server.